

Lean 4 Proof-Checked Delta-Hedging: Accounting Invariants with Empirical Validation Across Four Historical Regimes

EigenQ Research Series

Akhil Karra

May 23, 2026

Accounting bugs in backtesting systems are invisible to statistical validation: a ledger error that uniformly inflates the reported Sharpe ratio survives out-of-sample evaluation precisely because it applies to every period, in-sample and out-of-sample alike. This paper proves the ledger layer of an options backtester correct using machine-checked proofs (formal verification) in Lean 4 and pairs that guarantee with empirical validation on 491,390 WRDS OptionMetrics option-day observations across four historical market regimes. Three contributions follow. First, eleven accounting theorems covering NAV identity, cash-update correctness, quantity conservation, self-financing, option settlement, and payoff non-negativity are proved with zero unverified placeholders; the proofs hold unconditionally over all integer-representable inputs, not merely the inputs a test author chose to include. Second, the proved theorems compile to a native shared library that the Python backtester calls at runtime, so the same code object that carries the machine-checked guarantees is the code object that executes at runtime. Third, the formalization targets traditional-securities accounting invariants, a layer that prior proof-assistant work on financial systems has left unaddressed. Empirically, 500 seeded GBM paths yield a running-mean hedging cost converging to the Black-Scholes price with a 2.34% final deviation; the Black-Scholes implementation agrees with QuantLib to a maximum of 2.69 basis points across five scenarios. The accounting invariants hold unconditionally across all 491,390 WRDS observations with a 100% step-certificate pass rate; stress-regime hedging-cost medians are 1.69 (COVID 2020), 1.23 (GFC 2008 and Volmageddon 2018), and 1.18 (Q4 2018) as a fraction of the initial Black-Scholes premium. Primary results assume flat per-series implied volatility and European-style payoffs only; proportional transaction costs are left for future work.

1 Introduction

Every options backtester maintains an accounting ledger: cash balances are debited for rebalancing trades, option settlements are credited at expiry, positions are marked to the correct underlying price. When this ledger has a bug (a settlement dropped, a cash debit applied at the wrong step, a position quantity that drifts from its trade history), the resulting P&L series looks right. It generates a Sharpe ratio, it passes out-of-sample evaluation, it survives robustness checks across regimes. Yin et al. (2026) documents exactly this failure mode in a systematic audit of five widely used open-source backtesting engines; Löw, Maier-Paape, and Platen (2017) document the same class of silent error in a deployed production system. No amount of statistical testing can detect a ledger error that applies uniformly to every period: the test takes the reported P&L as given. The only reliable fix is to prove the ledger layer correct, so that errors of this kind become structurally impossible rather than merely unlikely.

Backtesting errors fall into two independent classes: accounting errors and model errors. Statistical overfitting has attracted sustained attention from empirical finance; Bailey et al. (2016) introduced combinatorially symmetric cross-validation to quantify this risk, Harvey, Liu, and Zhu (2016) found that most claimed findings in financial economics are likely false under conventional significance thresholds, and Jensen, Kelly, and Pedersen (2023) documented that overfitting persists even after out-of-sample testing is applied systematically. The second class (accounting errors) has received far less scrutiny: Löw, Maier-Paape, and Platen (2017) document this in production systems; Yin et al. (2026) provide a systematic taxonomy across five open-source engines. The distinguishing feature is that accounting bugs cannot be corrected by splitting the sample: the error is present in every period, so out-of-sample performance is just as inflated as in-sample performance. Extensive testing mitigates this risk but cannot eliminate it. A test suite checks correctness on inputs the test author thought to include, not on every input that live historical data will produce.

This paper proves the ledger layer of an options backtester correct in Lean 4 and pairs that proof with empirical validation across four historical market regimes.

Formal verification is a machine-checked proof that code behaves as specified for every possible input. A test suite checks ledger correctness on a finite set of trades the test author chose; a formal proof covers all integer-valued inputs simultaneously, with no dependence on the test author's choice of inputs. The consequence for ledger correctness: a bug that produces a wrong result only on untested inputs cannot be caught by testing, but is structurally ruled out in a proved system.

Three contributions: First, prior Lean 4 work on financial systems targets DeFi exchange mechanics (Pusceddu and Bartoletti (2024) and Dessalvi, Bartoletti, and Lluch-Lafuente (2026) apply Lean 4 to automated market makers); this paper formalizes the position ledger of a traditional-securities backtesting platform (the instrument-agnostic layer that tracks cash, positions, and NAV across any trading strategy). Second, prior Lean 4 financial proofs have no execution path to a running Python backtester; this paper bridges that gap via a compiled Python extension (Cython FFI — the interface that lets Python call compiled native code): the proved theorems compile to a

native shared library that the Python backtester calls at runtime, so the same code that carries the machine-checked guarantees is the code that executes. The closest structural antecedent is Natarajan, Sarswat, and Singh (2021), who extract certified OCaml from Coq proofs of exchange matching algorithms and run the extracted code against live NSE data; that work targets matching mechanics, not the accounting invariants of a running position ledger. Purpose-built industrial formal methods systems, such as Imandra Passmore and Ignatovich (2017), address exchange rule verification but are closed-source and target a different layer. Third, no machine-checked formalization of the operational position ledger invariants of a traditional-securities backtesting pipeline has appeared in the published literature reviewed here. The word operational distinguishes ledger correctness (cash, positions, net asset value) from pricing theory Echenim, Guiol, and Peltier (2021) and exchange mechanics Natarajan, Sarswat, and Singh (2021). The novelty lies in the conjunction: no prior work combines Lean 4, traditional-securities accounting invariants, and a proof that executes at runtime in Python via Cython FFI. One concession is required and must be stated plainly: the accounting invariants are proved correct for any integer-representable instrument; the Black-Scholes pricing model, the GBM path generator, and all statistical claims about hedging cost distributions are not and cannot be formally verified. Those layers are empirically validated in Sections 6–8 and cross-checked against QuantLib. The contribution is the verification workflow, not a novel accounting result for European vanilla options. European vanilla delta-hedging is tractable enough that a reader can check the paper’s accounting claims against Hull (2014) without proprietary access.

Lean 4’s stable release and the Mathlib4 port make this approach practical now: the LLVM backend produces a native shared library callable via Cython FFI Moura and Ullrich (2021).

Section 2 defines the discrete-time delta-hedging model and the position ledger; Section 3 states the eleven accounting theorems; Section 4 describes the proof artifacts and their scope; Section 5 describes the simulation and WRDS samples; Sections 6 and 7 report empirical validation and the four stress-regime audits; Section 8 states scope limitations. Full theorem statements, proof architecture, bootstrap specification tests, and multi-leg exhibits appear in the Appendix.

1.1 Related Work

The foundational financial contract DSLs (Peyton Jones, Eber, and Seward (2000) and Bahr, Berthold, and Elsman (2015)) formalize contract semantics as algebraic combinators. Annenkov and Elsman (2018) extend the Bahr (2015) algebra to a Coq proof of contract cashflow semantics. These papers establish that formal specification of financial operations is tractable but do not target ledger invariants for production backtesting pipelines.

The closest prior formalization of accounting invariants is Echenim, Guiol, and Peltier (2021), who prove ledger identities for the Cox–Ross–Rubinstein model in Isabelle/HOL, adopting the same generic/specific separation we use: instrument-agnostic portfolio accounting separate from option-specific pricing. Their work is a standalone Isabelle proof artifact with no path to Python execution. Natarajan, Sarswat, and Singh (2021) formalize exchange matching-engine invariants

in Coq, with extraction to OCaml and Haskell; Natarajan, Sarswat, and Singh (2024) extend this to automated checkers on real exchange data. Both are Coq-based and target exchange mechanics rather than backtesting accounting.

Passmore and Ignatovich (2017), (2020) introduced Imandra, a purpose-built industrial formal verification system deployed at real exchanges for matching-engine fairness and clearing-rule verification. Imandra combines SMT solving with ACL2-style induction. backtest-proofs differs in three ways: it uses Lean 4 and Mathlib4 (open-source, general-purpose, community-maintained) rather than a proprietary hybrid; it targets the accounting invariant layer of a backtesting pipeline rather than matching-engine rules; and it bridges to the Python practitioner ecosystem via Cython FFI rather than a proprietary exchange interface.

Prior Lean 4 work on financial systems targets **decentralized finance (DeFi)**: Pusceddu and Bartoletti (2024) apply Lean 4 to automated market makers, and Dessalvi, Bartoletti, and Lluch-Lafuente (2026) similarly. This paper operates in **traditional finance (TradFi)**: exchange-listed equity options under Black-Scholes delta hedging. The DeFi/TradFi distinction is material: smart-contract invariants concern token-reserve conservation under adversarial execution, while backtesting accounting invariants concern NAV identity and self-financing under sequential rebalancing.

Property-based testing Claessen and Hughes (2000) samples a finite set of inputs; the Lean 4 proofs here discharge universally quantified statements over all integer-representable inputs. A decade of documented replication failures Harvey, Liu, and Zhu (2016), Jensen, Kelly, and Pedersen (2023) motivates correctness guarantees that out-of-sample tests cannot supply.

Two bodies of quantitative finance literature bear directly on the empirical results. First, the discrete-hedging literature establishes the baseline against which the results here are evaluated: Figlewski (1989) measures hedging error across rebalancing frequencies and option characteristics; Hutchinson, Lo, and Poggio (1994) develop a nonparametric learning-network approach to pricing and hedging derivatives from market data, providing a model-free baseline for replication-cost estimation. The BKL and Leland tests in Section 6 reproduce and extend these baseline results on synthetic data verified by the proved position ledger. Second, the model risk management literature establishes that derivatives systems require machine-readable validation audit trails. SR 11-7 (Federal Reserve, 2011) and OCC Bulletin 2011-12 require that model validation include not only statistical backtesting but also conceptual soundness review and ongoing monitoring. The step-certificate architecture introduced in Section 2 produces a machine-readable audit trail at every rebalancing step, directly addressing the ongoing-monitoring requirement in those supervisory frameworks.

1.2 A Motivating Example

Consider a deliberately introduced accounting bug: the cash deduction for a delta-hedge trade is silently set to zero, a truncation error that would arise if a basis-point conversion function returned 0 instead of rounding to the nearest integer.

The trade-accounting theorem states that the change in portfolio value equals the mark-to-market gain on the *pre-trade* position minus the fee: $\Delta PV = q_{\text{before}} \times (p_{\text{exec}} - p_{\text{mark}}) - \text{fee}$. When the pre-trade quantity is zero (as it is on the first purchase step), this formula predicts $\Delta PV = 0$ regardless of the execution price. A test that only exercises zero-position steps cannot distinguish correct accounting from the broken version: the bug is invisible on those inputs. Any step with a non-zero pre-trade holding exposes the discrepancy immediately.

The listing below demonstrates the bug-invisible / bug-detected contrast:

```
from backtest_proofs.backtest.audit import verify_step

PV_BEFORE      = 1_000_000    # $100.00 in basis points
EXEC_PRICE_BP  = 500_000      # $50.00
MARK_BEFORE    = 490_000      # $49.00 (price moved up $1)
FEE_BP         = 0

# --- Case 1: pre_trade_qty = 0 (first purchase) ---
# BUG: pv_after = pv_before (cash not debited)
cert = verify_step(
    pv_before=PV_BEFORE, pv_after=PV_BEFORE,    # cash not debited
    pre_trade_qty=0, exec_price_bp=EXEC_PRICE_BP,
    mark_before_bp=MARK_BEFORE, fee_bp=FEE_BP, step=0,
)
assert cert.invariant_holds    # passes: trade accounting predicts dPV=0

# --- Case 2: pre_trade_qty = 500 (existing holding) ---
# trade accounting predicts: 500 x (500_000 - 490_000) = 5_000_000 bp
# BUG still produces pv_after = pv_before, so delta_pv = 0 != 5_000_000
verify_step(
    pv_before=PV_BEFORE, pv_after=PV_BEFORE,    # cash not debited
    pre_trade_qty=500, exec_price_bp=EXEC_PRICE_BP,
    mark_before_bp=MARK_BEFORE, fee_bp=FEE_BP, step=1,
)
# raises: ValueError: Accounting invariant violated at step 1:
# delta_pv=0 bp, expected=5000000 bp
```

The naive test (one that only checks steps where no position was held before the trade) passes, because the trade-accounting formula predicts zero change for a zero-size position regardless of the cash accounting. The step certificate, by contrast, records both the pre- and post-trade portfolio values and the formula prediction; when these are checked against the trade-accounting formula, the mismatch is detected unconditionally on any step where the pre-trade position is non-zero.

The executable exhibit is in `backtest-proofs/python/tests/test_backtest.py`, class `Test-`

Falsification.

2 The Backtesting System

2.1 The Delta-Hedging Strategy

To demonstrate the position ledger on a theoretically tractable instrument, the paper runs a discrete-time delta-hedging strategy through it. The strategy writer shorts N European call options at $t = 0$ and dynamically replicates the position over horizon T using n equally spaced rebalancing steps of width $\Delta t = T/n$. At inception the writer receives the Black-Scholes premium and opens a stock position of $\Delta_0 \cdot N$ shares at spot price S_0 . At each subsequent step $i \in \{1, \dots, n-1\}$ the hedger computes the new delta Δ_i from the current spot S_i and remaining time to expiry $T - i \cdot \Delta t$, and trades $(\Delta_i - \Delta_{i-1}) \cdot N$ shares at S_i . The cash account is debited (or credited) by the full cost of each rebalancing trade including any transaction fee. At expiry step n the option settles to $\max(S_n - K, 0) \cdot N$ in cash and the remaining stock position is liquidated at spot S_n .

The primary quantity of interest is the total hedging cost, defined as the initial premium received minus the terminal portfolio value:

$$\text{HedgingCost} = C_0 \cdot N - V_n,$$

where V_n is the portfolio value at expiry after settlement and unwinding. Under continuous rebalancing and geometric Brownian motion dynamics, the Black-Scholes replication argument guarantees $\text{HedgingCost} = 0$ almost surely. Under discrete rebalancing, residual variance from the gamma term generates a non-trivial hedging cost distribution whose mean and variance depend on step size Δt , volatility σ , and any transaction costs. The canonical deterministic example is Tables 19.2 and 19.3 of Hull (2014), which trace a 20-step weekly hedge for a written at-the-money call and tabulate the hedging cost under a specific realized path; Section 6 replicates this example as a deterministic cross-check before turning to Monte Carlo and empirical analysis.

The delta-hedging strategy is the demonstration vehicle. Every result in this paper that refers to ledger correctness is a property of the underlying position ledger, not of this specific strategy; any strategy that produces integer-valued trades would receive the same accounting guarantees.

2.2 The position ledger

The position ledger is implemented in `backtest-proofs/lean/BacktestProofs/Basic.lean`. It tracks three things: the portfolio's cash balance, its open positions, and the trades that update them.

A position is a named asset holding: an asset ID (a string), a signed integer quantity, and an integer mark price in basis points. A type-level constraint ensures the mark price is strictly positive; this is not a runtime check but an invariant enforced at construction time, so a position with a non-positive price is unrepresentable in the type system.

A portfolio collects positions alongside a cash balance. A type-level constraint (the NAV invariant)

ensures the portfolio's NAV always equals cash plus the sum of position values; the smart constructor discharges this proof obligation at compile time, making a portfolio with an inconsistent portfolio-value field unrepresentable. Every theorem that mentions portfolio value is ultimately a consequence of the NAV invariant.

A trade records a single order execution: an asset ID, a signed quantity delta, an execution price in basis points, and a fee. Proof fields make invalid trades unrepresentable: a trade with a non-positive execution price or a negative fee cannot be constructed without a corresponding proof. The trade-application function updates the position quantity, debits cash by the signed quantity times the execution price plus fee, and re-establishes the NAV invariant by reflection. No option-specific fields appear in any of these three types; the position ledger is entirely instrument-agnostic.

2.3 Integer basis-point representation

The position ledger stores all monetary values as integers in units of basis points (10^{-4} of one dollar; \$49.00 = 490,000 bp). The key point is that integer arithmetic is exact: the eleven accounting theorems hold as universal statements over all integer-valued inputs. The Cython extension passes integer values across the Python runtime bridge; when the Python layer computes a Black-Scholes price of \$1.2347 it calls `to_bp(1.2347)` to obtain 12,347 bp and passes that integer to Lean. The formal guarantees therefore attach to the same numbers the Python layer uses. The *position ledger* (Lean 4, integer arithmetic) carries machine-checked proofs and never operates on floating-point numbers; the *pricing and simulation layer* (Python, Black-Scholes, GBM) is empirically tested but not formally verified. The Python runtime bridge is the precise demarcation between these two layers.

2.4 The step certificate

At each rebalancing step the runner emits a step certificate recording whether the accounting invariant held for that specific trade. The certificate stores the pre- and post-trade portfolio values in basis points, the observed change in portfolio value, the change predicted by the trade-accounting formula ($\Delta PV = q_{\text{pre}} \cdot (p_{\text{exec}} - p_{\text{mark}}) - \text{fee}$), and a boolean pass/fail flag. If the flag is false, the runner raises an exception immediately with a diagnostic; a violation indicates a bug in the Python runtime bridge or Python bookkeeping, not a market event.

The step certificate is the connective tissue between the machine-checked layer and the falsification argument in Section 1.2. The certificate sequence for a complete backtest run provides a machine-readable audit trail: for every step across every path in the Monte Carlo sweeps and every WRDS regime window, the runner either confirms that the accounting invariant held or identifies the exact step and inputs where the invariant failed.

3 What the Proof Guarantees

The position ledger comes with eleven machine-checked theorems that describe exactly what the ledger cannot do wrong.

The first seven theorems concern the bookkeeping that every instrument shares: cash always updates by exactly the trade cost and fee; positions update by exactly the signed trade size; the net asset value always equals cash plus the sum of position values; a trade at the existing mark price changes NAV only by the fee. These seven theorems hold for any instrument representable as an integer mark price and a signed quantity: a stock leg, an equity futures contract, and an options position are formally indistinguishable at the ledger layer. The position ledger is instrument-agnostic by construction, not by convention.

The eighth theorem covers option settlement: at expiry, settling a European option changes the portfolio's NAV by the quantity held times the difference between the option's payoff and its pre-expiry mark price, a single equation that handles both in-the-money and out-of-the-money expiry paths without conditional logic at the call site. The remaining three theorems establish that call, put, and option payoffs are always non-negative, the economic statement that option holders bear no downside risk at expiry.

Table 4.1 maps each theorem to the accounting failure it prevents; Section 1.2 demonstrated with a concrete exhibit why formal proof catches a class of bugs that testing cannot.

4 Proof Artifacts and Scope

4.1 Lean 4 artifacts

Full proof architecture and Python runtime bridge details appear in the appendix.

Table 4.1: Lean 4 theorem map. All 11 theorems are machine-checked with zero proof placeholders. Full Lean identifiers and source locations are in Appendix A.

Theorem	What it guarantees	Accounting failure it prevents
NAV identity	Portfolio value always equals cash plus the sum of position values	Reported P&L that is inconsistent with the actual ledger entries
Cash update	Cash debited exactly by execution cost plus fee	Cash that appears or vanishes without a corresponding trade
Quantity conservation	Position size changes by exactly the signed trade quantity	Positions that drift from what was actually transacted
Trade accounting	NAV change equals mark-to-market gain on pre-trade holding minus fee	Rebalancing costs that are mis-attributed or omitted
Self-financing	A costless trade leaves NAV unchanged	Free-lunch errors where a zero-cost operation changes reported wealth
Self-financing with fee	NAV change under a fixed fee equals mark-to-market gain minus fee	Fixed transaction costs that are inconsistently applied
Well-formedness	A valid trade preserves portfolio type validity	Type violations that could corrupt downstream computations
Settlement	Option settlement changes NAV by quantity times (payoff minus mark)	Incorrect option expiry cash flows under ITM or OTM scenarios
Call payoff ≥ 0	Call option payoff is always non-negative	A call holder being charged at expiry
Put payoff ≥ 0	Put option payoff is always non-negative	A put holder being charged at expiry
Option payoff ≥ 0	Any option payoff is always non-negative	Payoff sign errors under any moneyness scenario

Formalization claim

At the version tagged for submission, every theorem in the position ledger is proved to completion: no proof contains an unverified placeholder, and the proof library compiles without errors. The full theorem statements with Lean 4 source identifiers are in Appendix A; the proof can be reproduced with `make verify-lean` in the repository.

4.2 Scope of formalization

The remaining four theorems in Table 4.1 (the settlement theorem from `BacktestProofs.SettlementInvariants` and three payoff theorems from `QuantCore.OptionInvariants`) are option-specific and do not generalize to other instruments without additional settlement modules and invariants. This design mirrors Echenim, Guiol, and Peltier (2021), who separate generic portfolio/market accounting from instrument-specific pricing in their Isabelle/HOL formalization of the Cox–Ross–Rubinstein model.

Instrument-class generality

The position ledger (NAV identity, self-financing, trade accounting) is formally verified for any instrument representable as an integer mark price and a signed quantity (stocks and equity futures included). Extending the *settlement* layer to equity futures (mark-to-market final settlement) or interest rate futures (accrued coupon) requires new Lean Settlement modules and corresponding invariants; that extension is left to future work.

The proved theorems bound what the ledger can do wrong; the empirical sections that follow test what the backtester actually produces on real and synthetic data.

5 Empirical Setup

5.1 Main sample

Table 5.1: Main sample characteristics.

Attribute	Value
Source	WRDS OptionMetrics (licensed; not committed)
Sample window	2019-01-01 to 2024-12-31
Universe	SPY, QQQ, AAPL, MSFT, JPM, IWM (6 tickers)
Frequency	Daily end-of-day snapshots
Moneyness filter	ATM $\pm 3\%$ ($ \text{S/K} - 1 \leq 0.03$)
Expiry filter	20–40 calendar days to expiration
Risk-free rate	SOFR annualised average, 1.65%
Final N (after filters)	491,390 option-day observations

These filters impose no restriction on the formal results; the Lean theorems hold for arbitrary strike prices and expiry dates. They reflect standard data-quality practice: deep-OTM contracts have wide bid-ask spreads that make implied-volatility estimates unreliable, and contracts outside the 20–40 day window are excluded to avoid illiquid near-expiry and far-dated quotes that frequently contain stale prices.

5.2 Stress-period samples

Four supplementary WRDS OptionMetrics downloads use the same schema and ticker universe (Table 5.2). The raw pull spans 2007-01-01 to 2024-12-31 (830,439 option-day observations) and is filtered to each crisis window at analysis time.

Table 5.2: Stress-period sample characteristics. COVID window uses the same portfolio_atm_options.parquet filtered to the 2020-02 to 2020-09 window.

Window	Dates	Regime	N observations
GFC peak	2008-09-01 to 2008-12-31	Liquidity crisis	527
Volmageddon	2017-12-01 to 2018-04-30	Volatility spike	5,060
Q4 2018 selloff	2018-09-20 to 2019-01-31	Rate-driven drawdown	25,090

Window	Dates	Regime	N observations
COVID crash	2020-02-01 to 2020-09-30	Pandemic shock	—

The GFC window (527 option-day observations) is substantially smaller than the others due to near-total illiquidity in ATM options during September–October 2008: bid-ask spreads widened to levels where implied-volatility estimates are unreliable, and most contracts fell outside the $\pm 3\%$ moneyness filter. These results warrant caution given the sparse GFC coverage.

5.3 Synthetic GBM and deterministic paths

The Monte Carlo convergence, BKL scaling, Carr-Madan, Leland, and QuantLib figures use no licensed data. GBM paths are seeded with `{python} MC_SEED` for full reproducibility.

Hull Tables 19.2 and 19.3 use the hardcoded weekly prices from Hull (2014) (Tables 19.2/19.3): $S_0 = \$49$, $K = \$50$, $r = 5\%$, $\sigma = 20\%$, $T = 20$ weeks.

6 Empirical Validation

Firm-internal evaluation of a quantitative backtester typically rests on three pillars: (i) deterministic replay against textbook or vendor reference trades, (ii) A/B testing the pricing layer against an independent industry-standard implementation, and (iii) statistical reconciliation of realized P&L against a Greek-based explained-P&L attribution. The Hull replication in Section 6.1 covers (i); the vendor cross-check below covers (ii). The Monte Carlo convergence, BKL scaling, and Carr-Madan decompositions in Section 6.2 through Section 6.5 are an empirical version of (iii).

6.1 Hull Table 19.2 / 19.3 replication

The simplest validation is the textbook's own deterministic example.

The Hull (2014) textbook tabulates a deterministic 20-step weekly hedge of a written ATM call ($S_0 = 49$, $K = 50$, $r = 5\%$, $\sigma = 20\%$, $T = 20$ weeks) under two realized price paths: Table 19.2 ends ITM at $S_T = 57.25$; Table 19.3 ends OTM at $S_T = 48.12$. Hull's printed cost-of-hedging figures (\$263,300 and \$256,600 respectively) incorporate textbook rounding conventions (delta to three decimal places, share counts to the nearest hundred, dollar amounts to the nearest hundred) that introduce noise of several thousand dollars on a ~\$250K hedging cost. The cleaner test is to recompute the hedge in Python using the *same* basis-point integer arithmetic that the Python runtime bridge uses, and compare the two NAV paths point by point. If the Python runtime bridge and an independent Python implementation of the same procedure agree to within a few basis points across every rebalance step, the Python runtime bridge is not introducing accounting drift on top of the proved kernel.

The two implementations agree to `{python} f"{hull_192_dev_bp:.2f}" bp` on the ITM path and `{python} f"{hull_193_dev_bp:.2f}" bp` on the OTM path. The remaining discrepancy is cumulative basis-point rounding through the FFI boundary; the noise floor for end-to-end cross-checks like this is approximately 1 bp per 20-step hedge. For comparison, Hull's printed cost-of-hedging figures sit roughly 3-4% above both implementations; the difference is fully explained by the textbook's three-decimal-place delta rounding and hundred-share rounding rules. All step certificates pass for both paths, and the certificate-by-certificate equality between the Python runtime bridge's reported delta-PV and the trade-accounting formula is verified inside the runner at every step (`?@fig-hull`).

6.2 Monte Carlo convergence

Across `{python} f"{MC_N_PATHS: ,}"` seeded GBM paths the running-mean hedging cost converges to the Black-Scholes price with a final deviation of `{python} f"{mc_mean_deviation_pct:.2f}"%` (`?@fig-mc`). The deviation lies inside the 3% tolerance band; the signed direction (cost below the BS price) is consistent with the $O(\Delta t)$ downward correction [Leland \(1985\)](#) under discrete rebalancing at $N=20$, and inconsistent with an accounting bug, which would produce a path-independent

bias whose magnitude does not shrink with rebalancing frequency.

6.3 BKL variance scaling

The empirical ratio $\text{std}(N=10) / \text{std}(N=20) = \{\text{python}\} \text{ f"\{bkl_ratio_10_20:.3f\}"}$ against the theoretical $\sqrt{2} \approx 1.414$ (?@fig-bkl). Across five rebalancing frequencies, the empirical slope of the std vs. $1/\sqrt{N}$ regression exceeds the BKL theoretical slope by approximately 13%; Boyle and Emanuel (1980) show that discretely adjusted hedges carry a residual variance above the continuous-time limit by a term proportional to the rebalancing interval, which accounts for this excess slope. The figure plots $\text{std}(\text{cost})$ against $1/\sqrt{N}$ on a linear scale; the log-log regression in Section 6.5 independently verifies the power-law exponent is 0.5, consistent with the linear-scale figure under exact BKL scaling.

The BKL scaling result is negative evidence for ledger correctness. A structural ledger error (an off-by-one in settlement, a mis-signed cash flow, a dropped option payoff) would produce a variance floor that does not shrink as rebalancing frequency increases. The $1/\sqrt{N}$ pattern here shrinks toward zero with N , ruling out any constant-magnitude accounting error. The 13% excess above the BKL theory slope is a finite- N correction from discrete rebalancing identified by Boyle and Emanuel (1980), not a floor: it contracts proportionally with the step interval, as the log-log regression in Section 6.5 confirms.

6.4 Carr-Madan gamma attribution

Hedging cost correlates with gamma P&L at $r = \{\text{python}\} \text{ f"\{carr_madan_corr:.3f\}"}$ across $\{\text{python}\} \text{ CM_N_PATHS}$ paths (?@fig-cm). The Carr-Madan decomposition Carr and Madan (1998), Carr and Wu (2020) predicts that realized cost lies on the 45-degree line against path-integrated gamma P&L; the observed r of $\{\text{python}\} \text{ f"\{carr_madan_corr:.3f\}"}$ means $\{\text{python}\} \text{ f"\{carr_madan_corr**2*100:.1f\}"}$ % of cost variance is explained by the gamma term alone, with the remaining $\{\text{python}\} \text{ f"\{(1-carr_madan_corr**2)*100:.1f\}"}$ % attributable to theta P&L, vega, and higher-order discretization terms Carr and Wu (2020).

The Carr-Madan scatter is also negative evidence for accounting correctness. A structural ledger error would shift points off the theoretical line in a pattern uncorrelated with gamma: if cash flows are mis-attributed or settlements dropped, the realized cost would carry a systematic component that the gamma term cannot explain. The tight linear scatter at $r = \{\text{python}\} \text{ f"\{carr_madan_corr:.3f\}"}$ is consistent with gamma-driven variance and inconsistent with any accounting error large enough to matter economically.

6.5 Error decomposition

The four measurements together characterize the error regime and distinguish discrete-hedge variance from structural bias.

Zero-bias test. A one-sample t-test on the relative error $(c_i - \text{BS})/\text{BS}$ across `{python} MC_N_PATHS` paths yields $t = \text{{python}} \text{ f}\{t_stat_bias:.2f\}$, $p = \text{{python}} \text{ f}\{p_val_bias:.4f\}$, with mean error `{python} f"{mean_err_pct:.2f}"` and standard deviation `{python} f"{std_err_pct:.2f}"`. The null of zero mean is rejected at the 1% level; the mean lies `{python} f"{abs(mean_err_pct)/std_err_pct:.2f}"` standard deviations below zero. The signed direction (negative: hedging cost below BS price) is consistent with the known $O(\Delta t)$ downward correction to the Black-Scholes price under discrete rebalancing [Leland \(1985\)](#), not with an accounting bug, which would produce a positive or path-dependent bias. The mean is economically small relative to the std (ratio `{python} f"{abs(mean_err_pct)/std_err_pct:.3f}"`). The t-test has asymptotic validity under the CLT at $n = 500$; a Wilcoxon signed-rank test on the same sample confirms the same directional conclusion (cost below BS price, $p < 0.01$).

Carr-Madan variance attribution. Across 200 GBM paths, the OLS regression of hedging cost on gamma P&L yields $R^2 = \text{{python}} \text{ f}\{cm_r_sq:.3f\}$, slope `{python} f"{cm_slope:.3f}"` (expected 1.0 under the Carr-Madan identity), and residual standard deviation `$`python f"cm_residual_std :.0f"``. The OLS slope is downward-attenuated relative to 1.0 by discretization error in the gamma P&L regressor (errors-in-variables bias); the R^2 is the more interpretable diagnostic. The R^2 of `{python} f"{cm_r_sq:.3f}"` means `{python} f"{cm_r_sq*100:.1f}"`% of hedging cost variance is explained by the one-factor gamma P&L term alone; the remaining `{python} f"{(1-cm_r_sq)*100:.1f}"`% reflects higher-order terms (theta P&L, vega, path-specific discretization noise) that the Carr-Madan single-factor model does not capture. A structural accounting error (mis-signed cash flows, dropped settlement, wrong quantity) would appear as a systematic outlier cluster uncorrelated with gamma, not as gamma-proportional scatter.

BKL log-log slope. A log-linear regression of $\log(\text{std})$ on $\log(1/N)$ across five rebalancing frequencies yields slope `{python} f"{bkl_ll_slope:.3f}"` (BKL theory: 0.5) with $R^2 = \text{{python}} \text{ f}\{bkl_ll_r_sq:.4f\}$. The near-perfect fit ($R^2 > 0.99$) confirms that variance grows exactly as $1/N$, the hallmark of independent per-step errors that wash out with hedging frequency. Structural errors (e.g., an off-by-one in settlement) would manifest as a floor that does not shrink with N .

Floating-point precision. The `to_bp / from_bp` conversions introduce at most 0.5 bp rounding per call. Over `{python} MC_N_STEPS` rebalancing steps with ~ 2 conversions per step, independent rounding accumulates to a standard deviation of approximately `$`python f"rounding_std_dollars :.0f"`` (`{python} f"{rounding_frac_of_ref:.2f}"`% of the BS price), which is `{python} f"{rounding_frac_of_std:.1f}"`% of the empirical hedging-cost standard deviation. Basis-point rounding contributes negligibly to observed variance.

6.6 Cross-validation against QuantLib

The standard reference for (ii) is QuantLib, an open-source C++ derivatives library widely used in quantitative finance. Comparing our basis-point integer Black-Scholes pricing to QuantLib's ana-

lytic pricer at matched inputs isolates the pricing layer from the position ledger: if the two pricers agree, the cost figures elsewhere in the paper cannot be attributed to a pricing-implementation bug. Practitioners reviewing this work will recognize the comparison as the standard “vendor cross-check” used by validation desks for new derivatives systems.

Across `{python} ql_n_scenarios` parameter scenarios the maximum deviation between our Black-Scholes implementation and QuantLib’s analytic pricer is `{python} f"{ql_max_dev:.2f}"` bp (**?@fig-ql**). All `{python} ql_n_scenarios` scenarios fall within a 3 bp tolerance (`{python} "yes" if ql_within_3bp else "no"`). Any systematic bias in the integer-arithmetic Black-Scholes implementation large enough to distort the hedging-cost figures elsewhere in the paper would appear as a diagonal offset here; the maximum 2.69 bp deviation rules out pricing bias at the 3 bp threshold used by buy-side validation desks.

The position ledger composes across multi-leg portfolios (straddle, bull call spread) and equity-only positions (covered call) with no changes to the Lean source: each leg routes through the same settlement and trade-update functions, so the proved theorems cover multi-leg portfolios by construction. Extended exhibits appear in the appendix.

The synthetic and textbook results above confirm the backtester works under controlled conditions; Section 7 tests whether those guarantees hold when the underlying data is genuinely adversarial.

7 Historical Stress Tests

7.1 WRDS 2019–2024 holdout

The WRDS OptionMetrics 2021–2023 holdout contains `{python} f"{wrds_holdout_n:," if wrds_holdout_n else "-"` qualifying call series drawn from the ATM $\pm 3\%$ filter applied to the full 2019–2024 panel, restricted to the 2021–2023 sub-period to avoid overlap with the COVID 2020 stress window. The median cost/premium in the 2021–2023 WRDS holdout is `{python} f"{wrds_holdout_median:.2f}" if wrds_holdout_median else "-"`, below 1.0 and consistent with the positive variance risk premium documented by Bakshi and Kapadia (2003): in the absence of a crisis, implied volatility at inception systematically exceeds realized volatility over the hedge horizon, so the hedger recovers more from the option premium than she spends on rebalancing. The below-1.0 median provides the complement to the four crisis panels: normal-market and crisis-market data together span the full distribution of cost/premium outcomes, with the position ledger certifying correctness in every case. Every step certificate passes across all `{python} f"{wrds_holdout_n:," if wrds_holdout_n else "all"` series; the settlement theorem holds for every ITM and OTM settlement encountered in the panel.

The four historical stress regimes below test whether the Python runtime bridge correctly invokes the proved kernel on data the synthetic GBM simulator cannot replicate: large abrupt price moves, sustained liquidity freezes, and instantaneous vol-regime changes. The position ledger either certifies every step certificate unconditionally or raises a violation immediately; there is no intermediate outcome.

7.2 Four historical stress regimes

Cost/premium ratios **above** 1.0 during the GFC 2008 (median 1.23)¹ and COVID 2020 (median 1.69; February 19 through March 23, 2020) windows reflect the crisis-period inversion of the variance risk premium: realized volatility far exceeded implied volatility at option inception, so the hedging cost exceeded the premium paid. This is the opposite of average conditions: Bakshi and Kapadia (2003) document that delta-hedged gains are systematically negative on average (implied vol > realized vol, cost/premium < 1.0), with the negative VRP most pronounced outside crisis periods, confirmed here by the WRDS 2021–2023 holdout (median 0.92; see Section 7.1). During GFC and COVID, the VRP reverses: realized vol spikes above the implied vol quoted when the position was opened. The accounting invariants proved in Table 4.1 hold regardless of whether cost/premium is above or below 1.0; the settlement theorem certifies the option settlement cash entry independently of the realized P&L level.

The Volmageddon window (January 22 through February 16, 2018; N=5,060 option-day observations, 527 qualifying call series; median cost/premium 1.23) captures a qualitatively different stres-

¹The GFC 2008 window has N=42 qualifying call series after the ATM $\pm 3\%$ filter and requiring ≥ 5 consecutive observations; OptionMetrics coverage for early-dated options in 2008 is sparse and these results warrant caution.

sor: an instantaneous implied-volatility discontinuity rather than a sustained liquidity freeze. The VIX, which had traded near 11 for much of January 2018, had already surged to approximately 17 by the close of Friday February 2; on February 5 it spiked intraday to near 50, closing above 37, among the largest single-day percentage spikes in the index's history to that date. Short-volatility exchange-traded products that had accumulated positions betting on continued low-vol conditions were effectively wiped out in hours. For the options positions in this study, the shock manifested as a realized volatility that briefly exceeded the implied volatility embedded in premia set at position inception. The window is short (25 calendar days), which limits the sample relative to GFC or the WRDS holdout, but this is precisely what makes it a clean stress test of the position ledger: the sudden jump in implied vol creates large, abrupt ledger entries that stress-test the trade-accounting formula under conditions of high price-sensitivity. All 527 series pass every step certificate, consistent with the unconditional character of the proved accounting invariants.

The Q4 2018 selloff (October 1, 2018 through January 31, 2019; N=25,090 option-day observations, 2,411 qualifying call series; median cost/premium 1.18) provides a third distinct regime type: a *rate-driven drawdown* in which implied volatility rose gradually rather than spiking. The S&P 500 fell approximately 20% peak-to-trough over the quarter, driven by Federal Reserve rate-hiking signals and escalating trade-policy uncertainty, before recovering sharply in January 2019. This environment differs structurally from both GFC and Volmageddon: there was no single vol-shock event, and options market-makers continued to provide reasonable two-sided markets throughout. The observed median of 1.18 (mildest of the four crisis panels, consistent with a gradual rather than sudden VRP reversal [Bakshi and Kapadia \(2003\)](#)) confirms the intuition that a rate-driven selloff compresses the VRP less severely than a vol-shock event. The larger sample (25,090 observations, 2,411 series) reflects the better market-access conditions relative to the GFC window. All 2,411 series pass every step certificate, with the settlement theorem holding across all ITM and OTM settlements.

Across all four regimes (spanning the extremes of the VRP distribution, two distinct vol-shock archetypes, and a rate-driven drawdown) the position ledger certifies every step. The formal guarantees from the settlement and trade-accounting theorems are unconditional: the Lean 4 proofs establish correctness for all valid inputs, not for a specific economic scenario. Whether realized volatility far exceeds implied vol (GFC, COVID, Volmageddon), roughly matches it (Q4 2018 with gradual repricing), or falls below it (normal conditions), the ledger identities hold by construction. The stress-regime results do not test the Lean proofs; they test whether the Python execution layer and the Python runtime bridge correctly invoke the proved kernel across data that the synthetic GBM simulator cannot replicate. Section 8 discusses what follows from these guarantees for scope and extension.

8 Discussion

8.1 What formal proofs guarantee, and what they do not

The Lean 4 proofs establish that the accounting invariants hold *by construction*, unconditionally, for all integer-representable inputs. Universal quantification over all inputs is categorically different from testing: the proofs cover every integer-representable input, not a sample of inputs, and the step-certificate mechanism makes the guarantee visible at runtime. The NAV identity, trade accounting, cash update, quantity conservation, and self-financing theorems are not statistical claims; they are universal quantifications over integer-valued inputs, discharged by Lean’s kernel at compile time. A user who encounters an unexpected P&L outcome can immediately rule out one entire class of explanation (accounting bugs), because the formal boundary makes that class impossible by construction.

However, the proofs do *not* establish that the Black-Scholes delta hedge converges to the option premium (a probabilistic statement about the pricing layer), that GBM accurately models equity price dynamics, or that the backtester’s P&L is economically meaningful in any specific market regime. Those claims are empirically supported in Sections 6–7 but remain outside the formal boundary. A user who finds that MC hedging cost deviates from the BS price by 15% can be certain the deviation is not from an accounting bug, but they cannot be certain it is not from model misspecification. The analogous point applies to the empirical volatility of hedging cost: the variance reported in the MC distributions reflects GBM path variance and Black-Scholes approximation error, neither of which is formally bounded. This separation of concerns (a proved accounting layer beneath an empirically validated pricing layer) is the paper’s core methodological claim.

This separation has practical implications for accounting oversight. Automated derivatives systems produce ledger entries at speeds that preclude manual inspection; an auditor or compliance function must accept the system’s output as given, or re-derive it independently. The step-certificate mechanism produces a machine-readable audit trail at every rebalancing step. The stress-regime results in Section 7.2 show that across all four crisis panels (sustained vol spikes, liquidity crises, instantaneous regime changes) every certificate passes unconditionally, confirming that the Python runtime boundary and Python bookkeeping layer do not introduce errors that the proved kernel would reject.

Deep hedging [Buehler et al. \(2019\)](#) optimizes the hedging *policy* via reinforcement learning. Formal verification certifies the accounting *ledger*. A portfolio running a deep-hedging policy and a proved position ledger is a coherent combination: the learning algorithm decides what to trade, the proved kernel guarantees those trades are recorded correctly.

8.2 Scope limitations

Three limitations bound the current results. First, the Black-Scholes implementation uses a *flat* implied volatility (the per-series inception-date IV) rather than a dynamic volatility surface. The

actual volatility smile means real hedging errors exceed the flat- σ prediction for in-the-money and out-of-the-money options, because Black-Scholes delta is computed under an incorrect local volatility when the smile is steep. Hull (2014), Ch. 20 documents that delta-hedging errors grow with the curvature of the volatility surface; El Karoui, Jeanblanc-Picqué, and Shreve (1998) establish that a flat implied-volatility hedge is a superhedge when the assumed volatility dominates the true local volatility, providing a one-sided formal bound on the hedging error. The Leland (1985) replication in Section 9.4 confirms a slope gap between theory and simulation that a flat-volatility model cannot close. Of the three limitations, this one has the largest quantitative effect: results in low-IV regimes (2021–2023) are more reliable than results in high-IV regimes (COVID 2020, Volmageddon 2018), where the smile is steepest.

Second, the settlement and payoff theorems are proved for European-style options only. American-style early exercise and barrier events require new settlement modules and new invariants: the settlement function would need to dispatch on an exercise boundary condition, and the corresponding Lean theorem would need to quantify over the early-exercise policy. These extensions are structurally straightforward but are not yet implemented.

Third, transaction costs are modeled as a fixed fee per trade (the fixed-fee self-financing theorem); stochastic bid-ask spreads and market impact are not modeled. The flat-fee model is conservative for large re hedge sizes and generous for small ones; a stochastic spread model would require a new trade field and a new theorem quantifying the expected transaction cost in terms of spread distribution parameters.

8.3 Extensions

Four extensions follow directly. First, the fixed-fee self-financing theorem takes a fixed integer fee per trade; extending to proportional transaction costs (e.g., bid-ask spread as a fraction of notional) would require a new lemma that binds the fee argument to $|qty| \times \text{spread} \times \text{executionPrice}$. With that lemma in place, one could connect the position ledger directly to the Leland (1985) modified volatility $\tilde{\sigma}^2 = \sigma^2(1 + \text{Le} \cdot \text{sgn}(\Gamma))$, yielding a formally verified replication-cost bound under proportional transaction costs.

Second, a Carr-Wu (2009) realized-vs.-implied volatility direction test (asking whether $\text{cost/premium} > 1$ is predictable from the IV-RV spread at inception) is a natural empirical extension that the position ledger supports without modification. The position ledger is agnostic to what drives hedging cost; step-certificate output contains all the cost-attribution data needed to regress cost/premium on the IV-RV spread as a predictor, testing the Carr-Wu hypothesis that overpriced implied volatility systematically inflates hedging cost.

Third, `ftap`-proofs and options-proofs (companion projects in this monorepo) target the discrete Fundamental Theorem of Asset Pricing Echenim, Guiol, and Peltier (2021) and put-call parity via the Cox-Ross-Rubinstein binomial model. A completed proof of either would add a formally verified pricing theory layer above the position ledger proved here, narrowing the gap between the formal boundary and the pricing claims left to empirical validation. The position ledger is silent

on whether delta-hedging is the optimal strategy or whether Black-Scholes is the right pricing model; it certifies only that whatever trades the strategy submits are recorded correctly.

Fourth, the mortgage-proofs project applies the same step-certificate pattern to LangGraph routing invariants in a multi-agent mortgage pipeline. A `DecisionRecord` in mortgage-proofs plays the same role as a step certificate in backtest-proofs: both record the pre- and post-decision state and check it against a Lean-proved invariant over integer-typed state variables.

The instrument-agnostic character of the position ledger extends beyond options to plain equity positions and covered calls, as demonstrated in the equity exhibits reported in the appendix: a position is a named asset with a signed integer quantity and an integer mark price, with no options-specific fields, so the trade-accounting and self-financing theorems certify equity step certificates and covered-call portfolios without proof modification.

8.4 Data Availability

Synthetic GBM results are fully reproducible from the public repository via `make paper` (base seed 20260519). WRDS OptionMetrics data is licensed and not redistributed per WRDS terms; the download query and instructions are in `backtest-proofs/docs/wrds_data_download.md`. All numerical results are recorded in `backtest-proofs/results/metrics.json`; the PDF regenerates deterministically from the QMD source when WRDS data is present.

9 Appendix

9.1 Theorem Statements

Full Lean 4 theorem statements, proof commentary, and source file references for all eleven machine-checked theorems are available in the public repository at github.com/eigenq-xyz/quant-proofs, in `backtest-proofs/lean/BacktestProofs/Invariants.lean`, `SettlementInvariants.lean`, and `quant-core/lean/QuantCore/OptionInvariants.lean`. The theorem names in Table 4.1 correspond directly to the Lean source identifiers; each hyperlinked row in the online version links to the exact line in the repository.

9.2 Proof Architecture

This section is a technical reference for developers wishing to inspect or extend the Python runtime bridge. QF/FM readers may skip to Appendix C.

The formal development compiles via `lake build` to a native shared library (`.so`) through Lean 4's LLVM backend. A Cython extension (`ffi/lean_ffi.pyx`) wraps the exported C symbols (`@[export hedge_*]`); `ffi/__init__.py` pre-loads `libleanrt` and `libuv` before importing the extension. Python calls the compiled kernel at runtime with no serialization overhead. All numeric values crossing the FFI boundary are basis-point integers ($\times 10,000$ of dollar values). See `backtest-proofs/DECISIONS.md` ADR-006 for the rationale behind the Cython FFI design choice.

9.3 Variance Scaling and Bootstrap Specification Test

Bertsimas, Kogan, and Lo (2000) and Leland (1985) predict specific relationships between discrete delta-hedge cost variance, mean bias, and rebalancing frequency; the exhibits below test those predictions. Both sets of results are placed in the appendix because they concern the behavior of the discrete-hedging model, not the correctness of the position ledger. The position ledger certifies every step at all frequencies; these exhibits are about finance theory.

9.4 Leland (1985) rehedg-frequencey sweep

Across $N=5$ to $N=80$, $\text{std}(\text{hedging cost})$ falls monotonically with rebalancing frequency, as Leland (1985) predicted. The empirical slope of the std vs. $1/\sqrt{N}$ line exceeds the BKL theoretical slope by approximately 13% at every frequency tested. Kabanov and Safarian (1997) prove that the limiting variance under Leland's modified volatility is strictly positive (contradicting Leland's original claim of zero asymptotic error); the 13% excess above the leading-order BKL slope is the measurable consequence of this nonzero term, not a defect in the backtester.

The mean relative bias decreases linearly toward zero as the rebalancing interval $\Delta t = T/N$ shrinks, with the OLS fit through the origin achieving $R^2 = \{\text{python}\} \text{ f"}\{\text{leland_bias_r_sq} : .3f\}$ " and

slope {python} f"{leland_bias_slope:.3f}" (?@fig-leland-bias). This empirical linearity is the $O(\Delta t)$ signature that Leland (1985) and Bertsimas, Kogan, and Lo (2000) predict: under discrete delta hedging of a written call, the gap between expected hedge cost and the Black-Scholes price contracts proportionally to T/N , not to $\sqrt{T/N}$ (which would indicate random noise) or to a constant (which would indicate a structural accounting bug). The sign is negative throughout (cost lies below the BS price), consistent with the well-known result that discrete replication underestimates the continuously-rebalanced Black-Scholes premium in the zero-transaction-cost case Leland (1985, sec. II). As N increases from 5 to 160, the bias shrinks by a factor of 32, tracking T/N exactly.

The $O(\Delta t)$ linear convergence rules out the two alternative explanations for the nonzero mean reported in §6: a structural accounting bug would produce a bias that does not converge to zero with N (the ledger error would be present regardless of rebalancing frequency), while stochastic path variance would produce a $O(1/\sqrt{N})$ decay in the mean, not an $O(1/N) = O(\Delta t)$ decay. The observed linear convergence with $R^2 > \{\text{python}\} f"{leland_bias_r_sq:.3f}"$ is consistent only with the discrete-replication interpretation. All step certificates pass for all 500 paths at each of the six frequencies, confirming that the accounting kernel certifies the ledger regardless of the realized direction of the bias.

9.5 Bootstrap specification test for BKL and Leland scaling

The point estimates above rely on OLS through the origin with R^2 as the goodness-of-fit diagnostic, a parametric criterion that assumes linearity and homoskedasticity. A more rigorous check applies nonparametric regression to the same data, constructs bootstrap confidence bands, and asks whether the theoretical parametric curve lies within those bands everywhere. The standard nonparametric regression specification test Shalizi (2013) fits a flexible smoother (a smoothing spline with automatic bandwidth selection) to the residuals of each parametric fit (BKL and Leland); if the smoother stays inside the bootstrap band around zero everywhere, the parametric form is plausible. If the smoother wanders systematically away from zero, the parametric form is missing some structure in the data.

The top panels of ?@fig-bootstrap-spec show a GCV-bandwidth smoothing spline nonparametric regression (blue curve) with $B=1,000$ case-resampling bootstrap 95% pointwise confidence intervals (shaded band), using $N=5,000$ paths at each of 20 log-spaced frequencies from $N=3$ to $N=400$ (100,000 simulations total). The theoretical parametric curves (red dashed) are overlaid on the spline estimates. The bottom panels apply the **specification test** Shalizi (2013): the smoothing spline applied to the parametric residuals (data minus theory), with the bootstrap band for those residuals. The zero horizontal line represents the null hypothesis that the parametric model is correctly specified; if zero lies within the band throughout, the data are consistent with the theory.

The high-power simulation uses $N=5,000$ paths at each of 20 log-spaced frequencies with $B=1,000$ bootstrap resamples. The BKL and Leland laws are *asymptotic* results: they describe leading-order behavior as $N \rightarrow \infty$ and $\Delta t \rightarrow 0$, and the extended simulation is sensitive enough to detect finite- N

corrections at the extremes of the frequency range.

BKL variance scaling (`{python} "PASS" if _bkl_passes else "FAIL" in N ∈ [20, 80]; {python} f"{_bkl_in_ci_frac:.0%}"` of all 14 frequencies within CI): The BKL leading-order law $\text{std} \propto C/\sqrt{N}$ has detectable systematic deviations even in the operationally relevant frequency range. The residual spline shows a positive arch in the $N \in [10, 80]$ region: the empirical std exceeds the OLS-through-origin prediction by a margin that the bootstrap CI cannot accommodate. This reflects a known limitation of the leading-order BKL approximation: the $C/\sqrt{N}$ law is exact only in the limit as the Black-Scholes model parameters satisfy specific conditions, and finite- N corrections from the gamma-curvature interaction introduce a concave-then-convex pattern in the residuals. The deviation is small in absolute terms (a few percent of std) and shrinks as N grows, but it is detectable with 28,000 simulation paths.

Leland $O(\Delta t)$ bias: The Leland linear-through-origin model fits within the bootstrap CI for `{python} f"{_leland_in_ci_frac:.0%}"` of frequencies, and passes the specification test over the practical range $N \in [20, 80]$ (`{python} "PASS" if _leland_passes else "FAIL"`). The residual panel shows that at very large Δt (small N), the mean bias is less negative than the leading-order $O(\Delta t)$ prediction: the extreme discrete paths partially recover toward the continuous limit due to a different balance between gamma-P&L and theta-P&L at very coarse grids. At very small Δt (large N), the mean bias is so small that individual path noise dominates, and the spline fit to residuals wanders within the wide CI.

The asymptotic laws are accurate for operationally relevant frequencies ($N \in [20, 80]$ corresponding to daily-to-weekly rebalancing) but show detectable finite- N corrections at extremes where higher-order theory applies. This is a stronger empirical claim than the earlier R^2 regressions, which had insufficient power (500 paths, 5–6 frequencies) to distinguish leading-order from next-order behavior. The position ledger certifies every step at all 14 frequencies and all 2,000 paths, regardless of which asymptotic regime the rebalancing frequency sits in.

9.6 Multi-leg Portfolios

The position ledger handles multi-leg portfolios without any proof modifications. The portfolio strategy runner routes every option leg through the same settlement function FFI call; the book-keeping loop is unchanged (only the number of positions grows), and step certificates are emitted for every rebalancing trade. The two exhibits below exercise both ITM and OTM settlement branches across both option types.

9.6.1 Short straddle: settlement decomposition

?@fig-straddle-settlement displays the per-leg payoff decomposition for a short straddle (short call + short put, both at $K = 50$, $N = 100K$ contracts) settled on the two Hull reference paths. On Hull 19.2 the stock closes at $S_T = 57.25$: the call expires in-the-money and transfers a positive payoff to the counterparty, while the put expires out-of-the-money and contributes nothing. On

Hull 19.3 the roles are exactly reversed: $S_T = 48.12$ leaves the put in-the-money and the call worthless. Together the two panels exercise both branches of `settlement_value_formula` (see Table 4.1): the ITM branch, which computes $\max(S_T - K, 0) \times N$ for calls and $\max(K - S_T, 0) \times N$ for puts, and the OTM branch, which returns zero. This is the minimal experiment that confirms the settlement dispatcher routes each leg correctly under `settlement_value_formula`. All step certificates pass for both paths.

9.6.2 Bull call spread

?@fig-bull-spread traces the portfolio value path for a bull call spread (long $K_{\text{long}} = 48$, short $K_{\text{short}} = 52$, $N = 100K$ contracts) on the Hull 19.2 path. Both calls expire in-the-money at $S_T = 57.25$: `settlement_value_formula` is invoked twice (once for the long leg with a positive quantity and once for the short leg with a negative quantity), and the net cash transfer is $(K_{\text{short}} - K_{\text{long}}) \times N = \$400K$. The portfolio value path in **?@fig-bull-spread** converges to the theoretical maximum payoff at expiry as the time value of the spread decays. The dashed green line marks the \$400K cap, and the terminal portfolio value equals it exactly. The per-step certificate panel (bottom) is uniformly green. No new Lean theorems were required: the accounting invariants proved for a single leg compose across legs by construction.

This correctness is **empirical**, not the consequence of a single proved multi-leg theorem: it follows from applying `settlement_value_formula` independently to each option leg in turn and summing the resulting settlement values, with no additional Lean proof required for the portfolio as a whole. This instrument agnosticism is a direct consequence of the position ledger’s design: because `PortfolioStrategy` strategy runner routes every leg through the same settlement function `FFI call`, and `settlement_value_formula` covers all combinations of option kind and moneyness, so multi-leg correctness requires no separate proof. The figures above test only written (short) legs in the straddle and a mixed long-short pair in the bull spread; a pure long position (bought call or put) was also exercised on the Hull reference paths and confirmed consistent with the same per-leg settlement application. No modification to the position ledger was required.

9.7 Equity and Covered-Call Demo

The `EquityStrategy` in the left panel of **?@fig-equity** holds 1,000 shares at a fixed quantity with no delta rebalancing. The portfolio value starts at exactly zero (the cost of purchasing the shares is funded by borrowing against the cash account, so the net asset value at inception is zero by `selfFinancing`) and then tracks $N \times \Delta S$ at each weekly step, with a small interest debit on the borrowed cash. Every step certificate passes, confirming that `valueUpdateFormula` applies to equity positions without any modification to the Lean kernel: the trade application function receives `(assetId, quantity, markPrice)` tuples with no instrument-specific fields, so an equity share and an option contract are formally indistinguishable at the ledger layer.

The `CoveredCallStrategy` in the right panel combines the same equity position with a short

ATM call ($K = 50$, $\sigma = 20\%$). The call premium received at inception raises the initial portfolio value above zero. At expiry ($S_T = 57.25 > K$), the call settles in-the-money and `settlement_value_formula` certifies the cash transfer; the equity leg has no settlement and contributes its final mark-to-market value to the terminal PV. All {python} `len(_cc_result.certificates)` step certificates pass across the covered-call path, and the terminal PV reflects the capped upside of the covered-call payoff: the equity gain above K is transferred to the call counterparty at settlement. No new Lean theorems were required for either the equity-only or the covered-call strategy: the existing `valueUpdateFormula`, `selfFinancing`, and `settlement_value_formula` compose across instrument types by construction.

9.8 Replication

```
git clone https://github.com/eigenq-xyz/quant-proofs
cd quant-proofs/backtest-proofs

# 1. Verify Lean proofs (Levels 1-2)
make verify-lean

# 2. Run Python tests (Levels 2-4)
make test

# 3. Install research dependencies and generate figures
cd python && uv sync --group research
cd ..
make paper
```

After downloading WRDS data per `docs/wrds_data_download.md`, re-run `make paper` to regenerate `?@fig-stress` with real market data.

9.9 Metrics snapshot

Numerical results are archived in `backtest-proofs/results/metrics.json`; the file is regenerated on each render.

References

- Annenkov, Danil, and Martin Elsmann, 2018, [Certified compilation of financial contracts](#), *Proceedings of the 20th international symposium on principles and practice of declarative programming (PPDP '18)* (ACM).
- Bahr, Patrick, Jost Berthold, and Martin Elsmann, 2015, [Certified symbolic management of financial multi-party contracts](#), *Proceedings of the twentieth ACM SIGPLAN international conference on functional programming (ICFP 2015)*.
- Bailey, David H., Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu, 2016, [The probability of backtest overfitting](#), *Journal of Computational Finance* 20, 39–69.
- Bakshi, Gurdip, and Nikunj Kapadia, 2003, [Delta-hedged gains and the negative market volatility risk premium](#), *Review of Financial Studies* 16, 527–566.
- Bertsimas, Dimitris, Leonid Kogan, and Andrew W. Lo, 2000, When is time continuous?, *Journal of Financial Economics* 55, 173–204.
- Boyle, Phelim P., and David Emanuel, 1980, Discretely adjusted option hedges, *Journal of Financial Economics* 8, 259–282.
- Buehler, Hans, Lukas Gonon, Josef Teichmann, and Ben Wood, 2019, [Deep hedging](#), *Quantitative Finance* 19, 1271–1291.
- Carr, Peter, and Dilip Madan, 1998, Towards a theory of volatility trading, *Volatility: New estimation techniques for pricing derivatives*, 417–427.
- Carr, Peter, and Liuren Wu, 2009, [Variance risk premiums](#), *Review of Financial Studies* 22, 1311–1341.
- Carr, Peter, and Liuren Wu, 2020, [Option profit and loss attribution and pricing: A new framework](#), *Journal of Finance* 75, 2271–2316.
- Claessen, Koen, and John Hughes, 2000, [QuickCheck: A lightweight tool for random testing of Haskell programs](#), *Proceedings of the fifth ACM SIGPLAN international conference on functional programming (ICFP 2000)*.
- Dessalvi, Marco, Massimo Bartoletti, and Alberto Lluch-Lafuente, 2026, A formal approach to AMM fee mechanisms with Lean 4, (arXiv:2602.00101).

Echenim, Mnacho, Hervé Guiol, and Nicolas Peltier, 2021, [Formalizing the Cox–Ross–Rubinstein pricing of European derivatives in Isabelle/HOL](#), *Journal of Automated Reasoning* 65, 669–714.

El Karoui, Nicole, Monique Jeanblanc-Picqué, and Steven E. Shreve, 1998, [Robustness of the Black and Scholes formula](#), *Mathematical Finance* 8, 93–126.

Figlewski, Stephen, 1989, [Options arbitrage in imperfect markets](#), *Journal of Finance* 44, 1289–1311.

Harvey, Campbell R., Yan Liu, and Heqing Zhu, 2016, [...And the cross-section of expected returns](#), *Review of Financial Studies* 29, 5–68.

Hull, John C., 2014, *Options, Futures, and Other Derivatives*. 9th Global. (Pearson).

Hutchinson, James M., Andrew W. Lo, and Tomaso Poggio, 1994, [A nonparametric approach to pricing and hedging derivative securities via learning networks](#), *Journal of Finance* 49, 851–889.

Jensen, Theis Ingerslev, Bryan T. Kelly, and Lasse Heje Pedersen, 2023, [Is there a replication crisis in finance?](#), *Journal of Finance* 78, 2465–2518.

Kabanov, Yuri M., and Mher M. Safarian, 1997, [On Leland’s strategy of option pricing with transactions costs](#), *Finance and Stochastics* 1, 239–250.

Leland, Hayne E., 1985, [Option pricing and replication with transactions costs](#), *Journal of Finance* 40, 1283–1301.

Löw, Robert, Stanislaus Maier-Paape, and Andreas Platen, 2017, Correctness of backtest engines, *Journal of Investment Strategies*.

Moura, Leonardo de, and Sebastian Ullrich, 2021, [The Lean 4 theorem prover and programming language](#), *Automated deduction — CADE 28*. Lecture notes in computer science.

Natarajan, Raja, Suneel Sarawat, and Abhishek Kr Singh, 2021, [Verified double sided auctions for financial markets](#), *12th international conference on interactive theorem proving (ITP 2021)*. Leibniz international proceedings in informatics (LIPIcs).

Natarajan, Raja, Suneel Sarawat, and Abhishek Kr Singh, 2024, [Double auctions: Formalization and automated checkers](#), *Journal of Automated Reasoning*.

Passmore, Grant O., Simon Cruanes, Denis Ignatovich, Johannes Kanig, Alain Mebsout, and Seyi Sheridan, 2020, [The Imandra automated reasoning system \(system description\)](#), *Automated reasoning — IJCAR 2020*. Lecture notes in computer science.

Passmore, Grant O., and Denis Ignatovich, 2017, [Formal verification of financial algorithms](#), *Automated deduction — CADE 26*. Lecture notes in computer science.

Peyton Jones, Simon, Jean-Marc Eber, and Julian Seward, 2000, [Composing contracts: An adventure in financial engineering](#), *Proceedings of the fifth ACM SIGPLAN international conference on functional programming (ICFP 2000)*.

Pusceddu, Daniele, and Massimo Bartoletti, 2024, Formalizing automated market makers in the Lean 4 theorem prover, *5th international workshop on formal methods for blockchains (FMBC 2024)*. OASICS.

Shalizi, Cosma Rohilla, 2013, *Advanced Data Analysis from an Elementary Point of View*.

Yin, Dong, Takeshi Miki, Vladislav Lesnichenko, and Vasyl Gural, 2026, [Implementation risk in portfolio backtesting: A previously unquantified source of error](#), (arXiv:2603.20319).